DOCUMENTACAO TECNICA CONSOLIDADA

Resenha

Documento mestre de arquitetura, dominio de negocio, API, aplicativo mobile, dados, testes e operacao. Elaborado a partir da leitura do codigo do workspace e da validacao dos artefatos tecnicos presentes no repositorio.

BASELINE ANALISADA
Workspace em 01/04/2026

ESCOPO
Backend, frontend, testes, scripts e deploy

AUDIENCIA
Engenharia, produto, operacao e onboarding tecnico

ARTEFATO FINAL
PDF consolidado para consulta e repasse

Escopo efetivamente lido
`backend/Resenha.API`, `backend/Resenha.API.Tests`, `frontend/resenha-app`, `backend/deploy/linux`, `scripts/` e a pasta `documentacao/` existente. Este documento privilegia o comportamento implementado no codigo-fonte atual, inclusive a feature de *desafio em andamento*, convidados por partida, historico detalhado, testes automatizados .NET e os fluxos mobile associados.

Sumario

1. Resumo Executivo
2. Escopo do Repositorio
3. Arquitetura da Solucao
4. Backend e API
5. Modelo de Dados
6. Fluxos e Regras de Negocio
7. Referencia de Endpoints
8. Aplicativo Mobile
9. Qualidade, Testes e Validacao
10. Infraestrutura e Operacao
11. Observacoes Tecnicas e Recomendacoes
12. Anexos Tecnicos

1. Resumo Executivo

O **Resenha** e uma plataforma para organizacao de peladas e grupos amadores de futebol. A solucao centraliza o que normalmente fica espalhado entre mensagens, planilhas e acordos informais: membros do grupo, convites, agenda, lista de presenca, definicao de capitaes, montagem dos times, encerramento da partida, classificacao e votacao de premios.

No estado atual, a solucao e composta por um **backend monolitico modular em ASP.NET Core 6**, um **aplicativo mobile em Expo/React Native**, persistencia em **MySQL**, deploy em **Oracle Cloud** com **nginx** e TLS, alem de artefatos operacionais e de teste que suportam regressao funcional e publicacao.

Capacidades ja implementadas

- Cadastro, login e sessao JWT com invalidacao por troca de senha.
- Recuperacao de senha com auditoria, expiracao, limite por IP e limite por usuario.
- Perfil do jogador com clube do coracao, posicao principal e pe dominante.
- Criacao de grupos, convites, aceite, recusa, reativacao de membership e gestao de papeis.
- Criacao de partidas, presenca, ausencia, convidados e reabertura de vaga com notificacao por e-mail.
- Ciclo de capitao, bloqueio de desafiantes derrotados e historico de desafios.
- Montagem dos times por desafio com par ou impar, picks alternados e etapa separada de goleiros.
- Finalizacao da partida, estatisticas individuais, classificacao da temporada e historico geral.
- Votacao de `MVP` e `BOLA_MURCHA` com rodadas de desempate.

Posicionamento tecnico atual

- Arquitetura adequada para beta controlado e operacao de pequeno porte.
- Regras de negocio concentradas nos servicos do backend, o que simplifica leitura e testes.
- Sincronizacao de estado do app baseada em chamadas REST e recarga de tela, sem WebSocket ou push.
- Deploy funcional em producao, porem ainda com divida operacional em observabilidade e automacao.
- Conjunto inicial de testes automatizados .NET ja cobrindo ciclo de capitao e desafio em andamento.

Conclusao executiva

O projeto ja ultrapassou a fase de prototipo raso: ha dominio esportivo modelado, consistencia transacional em fluxos criticos, artefatos de deploy e testes automatizados. O proximo salto de maturidade esta menos em criar modulos novos e mais em endurecer seguranca, telemetria, governanca de versao e automacao de entrega.

2. Escopo do Repositorio

O repositorio esta organizado por camadas e ja contem documentacao operacional, scripts de teste, templates de deploy Linux e um projeto de testes de backend. A tabela a seguir resume a responsabilidade de cada diretorio principal.

Diretorio	Papel no sistema	Observacoes relevantes
`backend/Resenha.API`	API REST principal, regras de negocio, EF Core, DTOs, entidades e bootstrap.	Contem controladores, servicos, helpers, SQLs utilitarios e configuracao de ambiente.
`backend/Resenha.API.Tests`	Testes automatizados .NET com `xUnit` e SQLite in-memory.	Foco atual em ciclo de capitao e desafio em andamento.
`backend/deploy/linux`	Templates de publicacao em Linux com `systemd`, `nginx` e ambiente.	Servem como baseline, mas parte da configuracao real de producao ja divergiu dos templates.
`frontend/resenha-app`	Aplicativo Expo/React Native consumindo a API.	Inclui navegacao, contexto de autenticacao, cliente HTTP, telas operacionais e build EAS.
`documentacao`	Capitulos em Markdown sobre produto, arquitetura, banco, operacao e backlog.	Esta documentacao consolidada reaproveita e valida esse material contra o codigo atual.
`scripts`	Scripts de regressao, seed e deploy complementar.	Inclui fluxos de votacao, capitao, recuperacao de senha e regresses especificas.

```
backend/
  Resenha.API/
  Resenha.API.Tests/
  deploy/linux/
frontend/
  resenha-app/
documentacao/
scripts/
Resenha.sln
```

Nota: o workspace analisado contem alteracoes locais ja presentes em codigo e documentacao. Este documento foi produzido com base nesse estado corrente, e nao apenas em arquivos previamente versionados.

3. Arquitetura da Solucao

3.1 Visao macro

A solucao adota um desenho pragmatico, centrado em um backend monolitico modular. Esse formato reduz overhead operacional, simplifica debugging e acelera evolucao funcional para o estagio atual do produto.

```
Aplicativo Android / Expo
  |
  | HTTPS + JSON + JWT Bearer
  v
api.resenhaapp.com
  |
  v
nginx (terminacao TLS / reverse proxy)
  |
  v
Resenha.API (ASP.NET Core 6)
  |
  v
MySQL
```

Componente	Responsabilidade primaria	Comentario arquitetural
App mobile Expo	Experiencia do usuario, sessao local, navegacao e consumo da API.	Estado da interface e recarregado por navegacao, refresh manual e pos-acao; nao ha canal realtime dedicado.
Resenha.API	Aplicar regra de negocio, autorizar acoes e persistir dados.	Concentra quase toda a inteligencia de dominio em servicos coesos por modulo.
MySQL	Persistencia transacional de usuarios, grupos, partidas, desafios, votacao e classificacao.	Indices unicos e validacoes de servico sustentam integridade funcional.
nginx	Receber trafego externo, encerrar TLS e encaminhar para a API interna.	Permite expor somente portas publicas necessarias e ocultar a porta do processo .NET.
Oracle Cloud VM	Hospedagem da API e do banco.	Modelo economico para beta, porem com menor isolamento entre aplicacao e banco.

3.2 Estilo de integracao

- **Cliente -> API:** HTTPS, JSON, timeout de 10 segundos, autenticacao JWT Bearer.
- **API -> banco:** Entity Framework Core com provider MySQL (`Pomelo.EntityFrameworkCore.MySql`).
- **Notificacoes por e-mail:** provider configuravel entre SMTP e SendGrid.
- **Deep linking mobile:** esquema `resenha://` para convite e redefinicao de senha.

3.3 Caracteristicas importantes observadas

- O backend ja considera o fuso `America/Sao_Paulo` para regras de janela de confirmacao.
- A montagem dos times e orientada por estado de dominio, nao por telas soltas.
- O app usa `useFocusEffect` e `RefreshControl` para sincronizacao; nao existe polling continuo nem socket.
- O design favorece consistencia funcional antes de otimizacoes de escala.

4. Backend e API

4.1 Stack e bootstrap

O backend esta em `.NET 6` e usa um bootstrap enxuto em `Program.cs`. Os pontos mais relevantes sao:

- Normalizacao da `DefaultConnection`, desabilitando SSL local quando o host e `localhost` e o provider estiver em `Preferred`.
- Registro do `ResenhaDbContext` com `ServerVersion.AutoDetect`.
- Autenticacao JWT com validacao de emissor, audiencia, assinatura, expiracao e claim customizada `pwd_at`.
- Revalidacao do usuario a cada token autenticado: usuario precisa existir, estar ativo e ter o mesmo timestamp de senha que originou o token.
- `CORS` permissivo em desenvolvimento e restrito por lista de origens em producao.
- Suporte a `X-Forwarded-For` e `X-Forwarded-Proto` para operacao atras de `nginx`.
- Resposta padronizada de `ModelState` invalido com payload `{ mensagem }`.
- Swagger habilitado apenas em desenvolvimento.

Nuance de seguranca relevante

A invalidacao de sessao por troca de senha esta bem resolvida pelo uso da claim `pwd_at`, porem o hash de senha atual usa apenas `SHA-256` sem salt dedicado nem algoritmo de custo adaptativo. Para producao madura, o correto e migrar para `Argon2`, `bcrypt` ou `PBKDF2`.

4.2 Controladores e modulos expostos

Controller	Dominio coberto	Pontos notaveis
`UserController`	Autenticacao, perfil, reset de senha e catalogo de clubes.	Possui proxy de logos de clubes e modo debug de token de reset apenas em desenvolvimento.
`GroupController`	Grupos, convites, membros, papeis, agenda e exclusao logica.	Ha dois estilos de convite: um endpoint simples por codigo e outro com tentativa de envio por e-mail.
`MatchController`	Partidas, presenca, convidados, historico, detalhes e desafio em andamento.	E hoje o controller mais rico do sistema, porque concentra o fluxo operacional da partida.
`CaptainController`	Ciclo de capitao, desafio, elegibilidade e resultado.	Modela o campeao da vez e a progressao de desafios no grupo.
`ClassificationController`	Montagem tradicional de times, finalizacao, ranking e estatistica pessoal.	Coexiste com a montagem via desafio; ambos desembocam no fechamento/classificacao.
`VoteController`	Votacao de premios da partida.	Suporta multiplas rodadas de desempate e aprovacao final pelo administrador.

4.3 Servicos de dominio

Servico	Escopo	Observacoes de implementacao
`AuthService`	Cadastro, login, perfil, troca de senha e recuperacao.	Normaliza e-mail, valida clube/posicao/pe dominante, audita reset e gera JWT.
`GroupService`	Grupos, convites, aceite, papeis, schedule e remocao.	Reativa membership antiga quando aplicavel e protege o ultimo admin do grupo.
`MatchService`	Partidas, presenca, convidados, historico, detalhes e desafio em andamento.	E o servico mais complexo; usa transacao serializavel em presenca e rollback de classificacao na exclusao.
`CaptainService`	Ciclo de capitao e historico de desafios.	Bloqueia desafiantes derrotados e registra historico da disputa por partida.
`ClassificationService`	Times, resultado, ranking da temporada, historico geral e estatistica pessoal.	Atualiza ranking por upsert e aplica pontuacao base por vitoria/derrota.
`VoteService`	MVP, bola murcha, desempate e aprovacao.	Usa rodadas sucessivas e restringe candidatos empatados por `Observacao`.
`InviteEmailService`	Envio de convites, reset e aviso de vaga reaberta.	Provider configuravel entre SMTP e SendGrid; falhas de e-mail nao bloqueiam o fluxo principal.

4.4 Tratamento de erros e contrato de resposta

O projeto utiliza uma extensao centralizada (`ApiErrorMapper`) para transformar mensagens de excecao em respostas HTTP. O payload de erro e simples e consistente: `{ mensagem: "..." }`.

- **404** quando a mensagem indica nao encontrado.
- **403** para violacao de membership, permissao administrativa ou convite pertencente a outro e-mail.
- **409** para conflitos como duplicidade, limite, lista cheia ou voto ja iniciado.
- **400** para demais erros de regra ou validacao.

> **Observacao pratica**
>
> O mapeamento e heuristico, baseado em fragmentos de texto de excecao. Funciona para o estagio atual, mas e um ponto natural de evolucao para erros tipados por codigo interno.

4.5 Autenticacao e identidade do usuario

- O cadastro exige `nome`, `email`, `senha`, `posicaoPrincipal` e `peDominante`.
- `Goleiro` pode ser marcado explicitamente ou inferido quando a posicao principal e `GOLEIRO`.
- O catalogo de clubes e local ao backend e retorna codigo, nome e URL do escudo.
- O endpoint `GET /api/users/clubs/logo` faz proxy apenas para hosts aprovados, evitando bloqueios em clientes moveis.

- A recuperacao de senha usa token hashado no banco, expiracao de 15 minutos e bloqueio apos mais de 5 tentativas de validacao.

5. Modelo de Dados

O `ResenhaDbContext` concentra o modelo relacional do sistema e define indices unicos que sustentam regras funcionais criticas. O desenho e orientado a agregados pequenos e claros, com estados codificados por strings.

5.1 Agrupamento das entidades

Dominio	Entidades	Finalidade pratica
Usuario e seguranca	`Usuario`, `TokenRecuperacaoSenha`, `AuditoriaSeguranca`	Identidade, credenciais, metadados esportivos e trilha de acoes sensiveis.
Grupo	`Grupo`, `GrupoUsuario`, `ConviteGrupo`	Governanca do grupo, membership, agenda e entrada por convite.
Temporada e partida	`Temporada`, `Partida`, `PresencaPartida`, `DesafioPartida`	Agenda, ocupacao de vagas e estado operacional do desafio da montagem.
Capitao	`CicloCapitao`, `CicloCapitaoBloqueado`, `HistoricoCapitao`	Persistencia do campeao do grupo, derrotados do ciclo e historico das disputas.
Times e resultado	`TimePartida`, `JogadorTimePartida`, `ResultadoPartida`, `EstatisticaPartida`	Composicao dos times, placar e estatisticas por jogador.
Votacao	`VotacaoPartida`, `Voto`	Rodadas de votacao, desempates e aprovacao final.
Classificacao	`ClassificacaoTemporada`, `ClassificacaoGeralGrupo`	Ranking anual e acumulado historico do grupo.

5.2 Indices de integridade mais relevantes

- `GrupoUsuario(IdGrupo, IdUsuario)`: impede membership duplicada no mesmo grupo.
- `PresencaPartida(IdPartida, IdUsuario)`: impede dupla presenca/ausencia para a mesma pessoa.
- `DesafioPartida(IdPartida)`: garante um unico estado de desafio por partida.
- `CicloCapitaoBloqueado(IdCiclo, IdUsuarioBloqueado)`: impede bloqueio duplicado.
- `HistoricoCapitao(IdPartida, IdCiclo)`: garante um registro historico consistente por disputa.
- `TimePartida(IdPartida, NumeroTime)`: protege os dois slots de time da partida.
- `JogadorTimePartida(IdTime, IdUsuario)`: impede o mesmo jogador duplicado no mesmo time.
- `ResultadoPartida(IdPartida)`: protege o placar unico da partida.
- `VotacaoPartida(IdPartida, Tipo, Rodada)`: garante unicidade de rodada por tipo de premio.
- `Voto(IdVotacao, IdUsuarioEleitor)`: impede voto repetido na mesma rodada.

- `ClassificacaoTemporada(IdTemporada, IdUsuario)` e `ClassificacaoGeralGrupo(IdGrupo, IdUsuario)`: base para upsert seguro de ranking.
- `ConviteGrupo(CodigoConvite)` e `Usuario(Email)`: unicidade de convite e identidade.

5.3 Estados persistidos

Objeto	Estados identificados no codigo
ConviteGrupo	`PENDENTE`, `ACEITO`, `CANCELADO`, `EXPIRADO`
Partida	`ABERTA`, `EM_ANDAMENTO`, `FINALIZADA`, `CANCELADA`
CicloCapitao	`ATIVO`, `ENCERRADO`
VotacaoPartida	`ABERTA`, `ENCERRADA`, `APURADA`, `APROVADA`
DesafioPartida	`AGUARDANDO_CONFIRMACOES`, `PRONTA_PARA_MONTAGEM`, `PAR_IMPAR_LINHA`, `ESCOLHA_EM_ANDAMENTO`, `PAR_IMPAR_GOLEIROS`, `ESCOLHA_GOLEIRO_EM_ANDAMENTO`, `DEFINICAO_MANUAL_GOLEIRO`, `TIMES_FECHADOS`

Observacao de modelagem

O dominio usa strings literais para varios estados e papeis. Isso facilita leitura inicial, mas aumenta o risco de drift entre backend, frontend, testes e documentacao. Uma evolucao natural seria consolidar essas constantes em enums ou catalogos compartilhados.

6. Fluxos e Regras de Negocio

6.1 Usuario, autenticacao e perfil

- Cadastro bloqueia e-mail duplicado e exige senha forte: minimo de 8 caracteres, ao menos 1 numero e 1 letra maiuscula.
- Posicao principal e pe dominante sao validados contra catalogos fixos.
- O token JWT carrega `pwd_at`; ao trocar a senha, tokens antigos deixam de ser validos.
- Recuperacao de senha aplica dois rate limits: por IP e por usuario.
- O reset invalida os demais tokens ainda nao usados daquele usuario.

6.2 Grupos e membership

- O criador do grupo entra automaticamente como `ADMIN`.
- Convites pendentes antigos para o mesmo e-mail e grupo sao cancelados antes de gerar um novo.
- O grupo respeita `LimiteJogadores` tanto na entrada por convite quanto na aceitacao de pendentes.
- Aceitar convite pode criar uma nova membership ou reativar uma antiga, sempre como `JOGADOR`.
- O sistema protege o ultimo administrador contra rebaixamento.

- Excluir grupo e exclusao logica: inativa o grupo, os membros e os convites pendentes.

6.3 Partidas, presenca, ausencias e convidados

- Somente administradores criam partida.
- Nao pode existir mais de uma partida nao cancelada por dia no mesmo grupo.
- Se nao houver temporada ativa, a criacao da partida abre automaticamente uma nova temporada.
- Confirmacao de presenca e adicao de convidado so podem ocorrer ate **20 minutos antes do horario da partida**.
- A confirmacao usa transacao serializavel para evitar overbooking concorrente nas ultimas vagas.
- Goleiros seguem regra especial: a partida aceita no maximo **2 goleiros confirmados**.
- Se a lista estava cheia e alguem cancela ou marca ausencia, o sistema tenta notificar os demais membros por e-mail sobre a vaga reaberta.
- Convidados sao persistidos como usuarios sinteticos com e-mail `@resenha.local`, sem ativacao de conta e vinculados a presenca da partida.

Detalhe operacional importante

O fluxo de convidados nao cria membership no grupo. O convidado existe para efeitos de presenca e composicao da partida, e pode ser limpo do banco quando a partida e removida e nao ha outras referencias para esse usuario sintetico.

6.4 Ciclo de capitao

- Um grupo so pode ter um ciclo `ATIVO` por vez.
- Para abrir ciclo, precisa haver temporada ativa.
- O admin que inicia o ciclo vira automaticamente o capitao atual.
- Somente o capitao atual escolhe o desafiante.
- O desafiante precisa estar confirmado na partida, nao pode ser convidado e nao pode estar bloqueado no ciclo.
- Se o capitao vence, o desafiante e bloqueado dentro daquele ciclo.
- Se o desafiante vence, o ciclo atual e encerrado e um novo nasce com ele como capitao.
- O historico de capitao registra a disputa por partida, com vencedor, derrotado e data.

6.5 Desafio em andamento e montagem dos times

Esta e a feature mais sofisticada do estado atual. O backend persiste o fluxo na entidade `DesafioPartida` e expoe um DTO rico, pensado para dirigir a UI do app sem logica duplicada no cliente.

- A montagem so e liberada com pelo menos **12 confirmados** e ao menos **1 goleiro confirmado**.
- Se houver apenas 1 goleiro, o sistema permite seguir, mas sinaliza que a definicao final do goleiro sera manual.
- O capitao atual escolhe `PAR` ou `IMPAR` para a etapa dos jogadores de linha.
- Cada capitao informa um numero de 0 a 10. Os valores ficam ocultos ate que ambos concluam o sorteio.

- O vencedor do par ou impar faz a primeira escolha; depois a vez alterna entre os capitaes.
- Os capitaes ja pertencem aos proprios times e nao entram na lista de escolha.
- Goleiros nao entram na fase de linha; eles sao tratados separadamente ao final.
- Quando restam 2 goleiros, o sistema reaproveita a paridade do sorteio principal para o sorteio dos goleiros.
- O vencedor da etapa dos goleiros escolhe um; o outro vai automaticamente para o time adversario.
- Se novos jogadores de linha confirmarem depois de os times estarem fechados, o fluxo pode voltar para `ESCOLHA_EM_ANDAMENTO`.

Nuance de implementacao

Embora a feature seja apresentada como desafio em andamento, a sincronizacao visual no app ainda e baseada em recarga de tela e refresh manual. Nao ha WebSocket, SignalR ou polling continuo. Em termos de produto, o comportamento e estado compartilhado com atualizacao por requisicao, e nao realtime estrito.

6.6 Encerramento da partida e classificacao

- Ha dois caminhos para chegar aos times: montagem tradicional por `AssignTeams` ou via desafio.
- A finalizacao so ocorre com a partida em `EM_ANDAMENTO` e sem empate.
- O sistema grava placar, estatisticas individuais e atualiza classificacao da temporada e geral.
- Pontuacao base: **4 pontos** para cada jogador do time vencedor e **1 ponto** para cada jogador do time perdedor.
- Todos os jogadores participantes recebem presenca; vencedores recebem vitoria e perdedores derrota.
- A exclusao de uma partida finalizada executa rollback de pontos, presencas, gols, assistencias e premios aprovados, sempre com clamp em zero.

6.7 Votacao de MVP e Bola Murcha

- A votacao so pode ser aberta apos a partida estar `FINALIZADA`.
- A abertura cria duas votacoes em paralelo: `MVP` e `BOLA_MURCHA`, ambas na rodada 1.
- Somente participantes confirmados podem votar, e nao e permitido votar em si mesmo.
- Empate nao encerra o processo: a rodada atual vira `ENCERRADA` e uma nova rodada aberta restringe os candidatos empatados.
- O premio so impacta ranking quando a rodada apurada e explicitamente aprovada por admin.
- Uma vez iniciada, a votacao da partida nao pode ser reaberta do zero.

---

7. Referencia de Endpoints

A seguir esta o contrato REST efetivamente identificado no codigo. Todos os endpoints abaixo devolvem JSON. Quando exigem autenticacao, o token deve ser enviado em `Authorization: Bearer <jwt>`.

7.1 Usuarios e autenticacao

Metodo	Rota	Auth	Finalidade
POST	`/api/users/register`	Nao	Cria conta e retorna JWT ja autenticado.
POST	`/api/users/login`	Nao	Autentica o usuario e devolve token.
POST	`/api/users/forgot-password`	Nao	Solicita reset de senha sem expor se o e-mail existe.
GET	`/api/users/reset-password/validate?token=...`	Nao	Valida se o token de reset ainda esta apto para uso.
POST	`/api/users/reset-password`	Nao	Redefine a senha usando token valido.
GET	`/api/users/clubs`	Nao	Lista clubes disponiveis para o perfil.
GET	`/api/users/clubs/logo?url=...`	Nao	Proxy seguro para carregar escudo do clube.
PATCH	`/api/users/profile`	Sim	Atualiza nome, clube, posicao, pe dominante e flag de goleiro.
PATCH	`/api/users/change-password`	Sim	Troca senha autenticada e ja devolve um novo token.

7.2 Grupos

Metodo	Rota	Auth	Finalidade
POST	`/api/groups`	Sim	Cria grupo e ja inclui o criador como admin.
GET	`/api/groups/me`	Sim	Lista grupos ativos do usuario.
POST	`/api/groups/{id}/invite`	Sim	Cria convite simples por codigo para um e-mail.
POST	`/api/groups/join`	Sim	Entra no grupo usando codigo de convite.
GET	`/api/groups/invites/pending`	Sim	Lista convites pendentes do usuario logado.
POST	`/api/groups/invites/{inviteId}/accept`	Sim	Aceita convite pendente.
POST	`/api/groups/invites/{inviteId}/reject`	Sim	Recusa convite pendente.
PATCH	`/api/groups/{id}/schedule`	Sim	Atualiza dia da semana e horario fixo do grupo.
GET	`/api/groups/{id}/members`	Sim	Lista membros ativos do grupo.

Metodo	Rota	Auth	Finalidade
GET	`/api/groups/{id}/invites/pending`	Sim	Lista convites pendentes daquele grupo.
POST	`/api/groups/{id}/members`	Sim	Convida membro por e-mail e tenta disparar comunicacao.
DELETE	`/api/groups/{id}/members/{memberUserId}`	Sim	Remove membro do grupo.
PATCH	`/api/groups/{id}/members/{memberUserId}/role`	Sim	Promove ou rebaixa papel do membro.
DELETE	`/api/groups/{id}`	Sim	Exclusao logica do grupo.

7.3 Partidas e desafio em andamento

Metodo	Rota	Auth	Finalidade
POST	`/api/matches`	Sim	Cria partida e temporada ativa quando necessario.
GET	`/api/groups/{groupId}/matches`	Sim	Lista partidas com status de presenca, ausencias e vencedores.
GET	`/api/groups/{groupId}/matches/history`	Sim	Historico resumido das partidas do grupo.
GET	`/api/matches/{id}/details`	Sim	Detalhe completo da partida, com placar, times, estatisticas e premios.
GET	`/api/matches/{id}/challenge-status`	Sim	Snapshot completo do desafio em andamento para a UI.
POST	`/api/matches/{id}/challenge/line-draw/start`	Sim	Capitao atual escolhe `PAR` ou `IMPAR`.
POST	`/api/matches/{id}/challenge/line-draw/number`	Sim	Capitao informa numero de 0 a 10 na etapa de linha.
POST	`/api/matches/{id}/challenge/line-picks`	Sim	Capitao da vez escolhe jogador de linha.
POST	`/api/matches/{id}/challenge/goalkeeper-draw/start`	Sim	Reinicializa a etapa dos goleiros reutilizando a paridade principal.
POST	`/api/matches/{id}/challenge/goalkeeper-draw/number`	Sim	Capitao informa numero na etapa dos goleiros.
POST	`/api/matches/{id}/challenge/goalkeeper-pick`	Sim	Capitao vencedor escolhe um goleiro; o outro e alocado automaticamente.

Metodo	Rota	Auth	Finalidade
POST	`/api/matches/{id}/confirm`	Sim	Confirma presenca do usuario logado.
DELETE	`/api/matches/{id}/confirm`	Sim	Cancela presenca confirmada.
POST	`/api/matches/{id}/absent`	Sim	Marca ausencia.
DELETE	`/api/matches/{id}/absent`	Sim	Remove ausencia.
DELETE	`/api/matches/{id}`	Sim	Exclui partida e dados relacionados, com rollback quando necessario.
POST	`/api/matches/{id}/guests`	Sim	Admin adiciona convidado diretamente a lista da partida.

7.4 Ciclo de capitao

Metodo	Rota	Auth	Finalidade
POST	`/api/groups/{groupId}/captain/draw`	Sim	Inicia ciclo de capitao.
GET	`/api/groups/{groupId}/captain`	Sim	Le status atual do ciclo.
POST	`/api/groups/{groupId}/captain/challenge`	Sim	Capitao escolhe desafiante para uma partida.
GET	`/api/groups/{groupId}/captain/eligible/{matchId}`	Sim	Lista elegiveis confirmados e nao bloqueados.
POST	`/api/groups/{groupId}/captain/result`	Sim	Admin registra vencedor do desafio.

7.5 Classificacao e votacao

Metodo	Rota	Auth	Finalidade
POST	`/api/matches/{id}/teams`	Sim	Monta manualmente os dois times.
POST	`/api/matches/{id}/finalize`	Sim	Encerra a partida e atualiza ranking.
GET	`/api/groups/{groupId}/classification`	Sim	Ranking da temporada ativa.
GET	`/api/groups/{groupId}/classification/all-time`	Sim	Ranking historico do grupo.
GET	`/api/groups/{groupId}/classification/me`	Sim	Estatistica do usuario logado (temporada + geral).
POST	`/api/matches/{id}/vote/open`	Sim	Abre votacao de premios.

Metodo	Rota	Auth	Finalidade
POST	`/api/matches/{id}/vote`	Sim	Registra voto.
POST	`/api/matches/{id}/vote/close`	Sim	Fecha rodada e apura ou abre desempate.
POST	`/api/matches/{id}/vote/approve`	Sim	Aprova vencedor provisorio e grava premio no ranking.
GET	`/api/matches/{id}/vote`	Sim	Consulta estado atual e historico das rodadas.

8. Aplicativo Mobile

8.1 Stack e configuracao

- `Expo 54`, `React Native 0.81`, `React 19`, `TypeScript`.
- Navegacao com `@react-navigation/native` e stack nativa.
- Persistencia de sessao com `AsyncStorage`.
- HTTP com `axios`, timeout de 10 segundos e interceptor para injetar JWT.
- Variavel principal de ambiente: `EXPO_PUBLIC_API_BASE_URL`.
- Fallback local por plataforma quando a variavel nao esta definida.

8.2 Navegacao

O app mantem dois stacks:

- **Auth stack:** login, cadastro, esqueci a senha e reset de senha.
- **App stack:** home, grupos, partidas, capitao, votacao, classificacao, perfil e historico.

O deep linking esta configurado para `resenha://`, o que conversa diretamente com os templates padrao de convite e reset gerados pelo backend.

8.3 Contexto de autenticacao

- Restaura sessao persistida ao abrir o app.
- Login e registro escrevem `@resenha:token` e `@resenha:user`.
- O registro pode tentar entrar automaticamente em um grupo quando a tela e aberta por convite.
- Troca de senha atualiza o token local imediatamente.
- Reset de senha forca logout local apos sucesso.

8.4 Telas principais e papel funcional

Tela	Papel	Chamadas principais
`HomeScreen`	Lista grupos e convites pendentes; tambem permite aceitar, recusar e excluir grupo.	`/api/groups/me`, `/api/groups/invites/pending`
`CreateGroupScreen`	Criacao do grupo com nome, descricao, limite e agenda base.	`POST /api/groups`
`JoinGroupScreen`	Entrada manual por codigo de convite.	`POST /api/groups/join`
`GroupDashboardScreen`	Centro operacional do grupo: partidas, presenca, ausencia, convidados, agenda e acesso as demais areas.	`/matches`, `/captain`, `/groups/me`, confirm/absent/guests
`CaptainScreen`	Acompanha o ciclo de capitao, abre desafio e registra resultado.	`/captain`, `/captain/eligible`, `/captain/challenge`
`DesafioEmAndamentoScreen`	Mostra o snapshot do desafio, picks, goleiros, alertas e bloqueios.	`/challenge-status` e endpoints de pick/sorteio
`CreateMatchScreen`	Abre nova partida para o grupo.	`POST /api/matches`
`ClassificationScreen` / `MyPerformanceScreen`	Exibem ranking coletivo e visao individual do usuario.	`/classification`, `/classification/all-time`, `/classification/me`
`VoteScreen`	Opera votos, desempates e aprovacao.	`/vote`, `/vote/open`, `/vote/close`, `/vote/approve`
`HistoricoPartidasScreen` / `DetalhePartidaScreen`	Consultam historico e detalhe completo da partida.	`/matches/history`, `/matches/{id}/details`
`ManageMembersScreen`	Lista membros, convites pendentes, remocao e alteracao de papel.	`/members`, `/invites/pending`, `/role`
`ProfileScreen` / `ChangePasswordScreen`	Perfil esportivo do usuario e seguranca da conta.	`/api/users/profile`, `/api/users/change-password`

8.5 Linguagem visual

O app usa uma identidade visual escura, com base verde/preta e acento neon-lime. O tema concentra cores, raios, espacamentos e tipografia em um unico modulo, facilitando consistencia visual entre telas.

8.6 Nuances de implementacao importantes

- O frontend sabe diferenciar goleiros na lista de presenca e aplica bloqueio visual ao terceiro goleiro.
- O dashboard do grupo ja expoe um banner para navegar ao desafio em andamento quando ha capitao e desafiante.
- A experiencia de desafio ja foi desenhada com flags de permissao devolvidas pelo backend, o que reduz logica condicional no app.
- Ha um desalinhamento de nomenclatura no tipo de grupo do frontend: o union local ainda admite `MEMBRO`, enquanto o backend usa `JOGADOR`.

---

9. Qualidade, Testes e Validacao

9.1 Testes automatizados .NET

O repositorio ja possui um projeto formal de testes em `backend/Resenha.API.Tests`, baseado em `xUnit`, `Microsoft.NET.Test.Sdk` e SQLite in-memory para isolar comportamento de dominio.

Arquivo de teste	Cobertura principal
`CaptainServiceTests.cs`	Criacao do ciclo, criterios de elegibilidade, bloqueio do derrotado, transicao de ciclo e ordenacao de historico.
`MatchServiceChallengeTests.cs`	Limite de goleiros, convidados, janela de 20 minutos, notificacao de vaga reaberta, par ou impar, picks, goleiros, exclusao da partida e retomada do fluxo.

Validacao executada durante esta producao

Foi executado `dotnet test backend/Resenha.API.Tests/Resenha.API.Tests.csproj --nologo --verbosity minimal` no workspace atual em **01/04/2026**, com resultado: **25 testes aprovados, 0 falhas, 0 ignorados**.

9.2 Scripts de regressao e operacao

Alem dos testes .NET, o projeto mantem scripts PowerShell para fluxos ponta a ponta e regresses especificas:

- `scripts/test-vote-flow.ps1`
- `scripts/test-captain-flow.ps1`
- `scripts/test-password-recovery-flow.ps1`
- `scripts/test-delete-match-classification-regression.ps1`
- `scripts/test-regression.ps1`

Estes scripts nao foram reexecutados nesta producao do PDF. O status verificado diretamente aqui foi apenas o da suite .NET.

9.3 Leitura de maturidade de qualidade

- O dominio mais sensivel ja tem testes de servico que exercitam regra, nao apenas controller.
- A combinacao de testes .NET e scripts de fluxo e um bom ponto de partida para um pipeline CI.

- Ainda vale ampliar cobertura para `AuthService`, `GroupService`, `ClassificationService` e `VoteService`.

10. Infraestrutura e Operacao

10.1 Estado de publicacao identificado

- API publica em `https://api.resenhaapp.com`.
- Hospedagem em VM Linux na Oracle Cloud.
- `nginx` na borda com TLS e proxy para a API interna.
- MySQL local na mesma VM.
- Servico gerenciado por `systemd`.

10.2 Variaveis e integracoes externas

Grupo de configuracao	Exemplos	Uso
Conexao e JWT	`ConnectionStrings__DefaultConnection`, `JwtSettings__SecretKey`, `JwtSettings__Issuer`, `JwtSettings__Audience`	Persistencia, assinatura de tokens e identidade da API.
Convite e reset	`InviteSettings__RegisterLinkTemplate`, `PasswordResetSettings__ResetLinkTemplate`	Deep links enviados por e-mail.
E-mail	`EmailSettings__Provider`, `EmailSettings__Host`, `EmailSettings__SendGrid__ApiKey`	Convites, reset de senha e aviso de vaga reaberta.
Frontend	`EXPO_PUBLIC_API_BASE_URL`	Base URL consumida pelo app mobile.

10.3 Operacao Linux

Os templates de deploy do repositorio apontam para um cenario `dotnet Resenha.API.dll`, mas a documentacao operacional existente indica que a publicacao atual roda como binario self-contained em `/opt/resenha/api/current/Resenha.API`, servindo internamente em `127.0.0.1:5000`.

> **Ponto de atencao**
>
> Ha divergencia entre o template versionado e a forma efetiva de execucao em producao. Em termos de manutencao, isso pede alinhamento do template ou atualizacao explicita do runbook para evitar deploys inconsistentes.

10.4 Build do app Android

- Projeto Expo ja vinculado ao EAS por `projectId`.
- Perfil `preview`: gera APK para distribuicao interna.

- Perfil `production`: gera AAB para futura publicacao.
- Comando pratico: `npx eas-cli build -p android --profile preview`.

10.5 Checklist operacional minimo

- Validar `systemctl status resenha-api`, `nginx` e `mysql`.
- Confirmar `curl https://api.resenhaapp.com/api/groups/me` retornando `401` sem token.
- Garantir que a porta 3306 do banco nao esteja exposta publicamente.
- Revisar segredos fora do repositorio, preferencialmente em arquivo de ambiente da VM.

---

11. Observacoes Tecnicas e Recomendacoes

11.1 Pontos fortes observados

- Boa separacao por dominio no backend, com servicos legiveis e regras centralizadas.
- DTO do desafio em andamento e rico e adequado para dirigir a UI mobile sem logica duplicada.
- Rollback de classificacao na exclusao de partida finalizada e raro em produtos similares e foi tratado com cuidado.
- Ja existe embriao de estrategia de testes automatizados em camada de servico.
- Os fluxos de convite, reset de senha e deep linking estao coerentes entre backend e app.

11.2 Recomendacoes prioritarias

1. **Seguranca de senha:** migrar o hash atual de `SHA-256` para algoritmo apropriado de password hashing.
2. **Observabilidade:** adicionar logging estruturado mais amplo, health checks e monitoramento basico de producao.
3. **CI/CD:** automatizar build, testes .NET, lint do app e publicacao controlada.
4. **Consistencia de tipos/estados:** unificar papeis e estados entre backend, frontend, testes e documentacao.
5. **Realtime verdadeiro:** se a montagem simultanea dos times for critica para o produto, considerar WebSocket/SignalR ou polling controlado.
6. **Infra:** alinhar templates versionados de `systemd` e `nginx` com a forma real de publicacao em producao.

11.3 Observacoes especificas da feature de desafio

- O DTO expoe `UsuarioPodeEscolherParidadeGoleiro`, mas a implementacao atual reaproveita a paridade principal e nao habilita uma escolha nova pelo usuario.
- O status `ESCOLHA_GOLEIRO_EM_ANDAMENTO` ja existe no backend e nos testes, e precisa constar sempre em documentacao e clientes.
- O fluxo ja reage a entrada tardia de novos jogadores de linha, retomando a etapa de picks se necessario.

---

12. Anexos Tecnicos

12.1 Catalogos de perfil esportivo

- **Posicoes principais validas:** `GOLEIRO`, `ZAGUEIRO`, `LATERAL`, `VOLANTE`, `MEIA`, `PONTA`, `ATACANTE`.
- **Pe dominante valido:** `DIREITO`, `ESQUERDO`, `AMBIDESTRO`.

12.2 Deep links padrao

- **Convite:** `resenha://register?email={email}&invite={code}`
- **Reset:** `resenha://reset-password?token={token}`

12.3 Artefatos tecnicos que merecem consulta complementar

- `backend/Resenha.API/sql/full-migration.sql` e scripts incrementais em `backend/Resenha.API/sql/`
- `documentacao/08-regras-de-negocio.md` e `documentacao/13-desafio-em-andamento.md`
- `scripts/test-regression.ps1` e scripts relacionados
- `backend/deploy/linux/*` para baseline operacional

Documento consolidado gerado a partir da leitura do codigo do workspace em 01/04/2026. Quando endpoints, regras, estados ou infraestrutura mudarem, o ideal e atualizar este artefato junto com o codigo, preservando-o como documento de onboarding e referencia operacional.